claude-windows / 75d3d121-f240-4293-afa5-b436a379ad31

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-khelsutra-guru-badminton-highlight-indexer\75d3d121-f240-4293-afa5-b436a379ad31\subagents\workflows\wf_5f13fc79-bad\agent-a33f5cc7c4685f37d.jsonl`
- SHA-256: `869d6982f591d104c814df4c88b3fe54e8bc58bb2926bc3bef55665b48bdc4e9`
- Source modified: `2026-07-08T06:19:08+00:00`
- Imported at: `2026-07-08T15:59:27+00:00`
- project: `wf_5f13fc79-bad`
- session_id: `75d3d121-f240-4293-afa5-b436a379ad31`

Transcript

1. user (2026-07-08T06:14:14.435Z)

CONTEXT: This is GitHub issue #521 (Khelsutra/badminton-highlight-indexer), MERGED as PR #525. It makes pipeline phase?machine placement config-driven and dispatches the reel compile/stitch to
an L4 Cloud Run worker (NVENC), mirroring the existing detection dispatch.

The full unified diff is at: undefined (read it first).
The full post-change source is under: undefined/ (read whole files there for context ? e.g. undefined/backend/orchestration.py, undefined/backend/worker/__main__.py, undefined/backend/compute/cloud_run.py, undefined/backend/config/placement.py, undefined/backend/config/models.py, undefined/backend/pipeline/stitcher/compiler.py).

Key design invariants that MUST hold (a violation is a real bug):
- DEFAULT-OFF: with no phase_placement config and compute_target unset, behaviour is byte-identical
  to before #521 (validation+compile on control-plane, detect in-process).
- The cloud compile path must return the SAME {status, filepath, download_url, video_id} shape the
  SPA already polls for; a stitch failure must be a clean failed-job, never a crash.
- The worker 'compile' subcommand reads a bucket-staged spec (resolved time-pairs + stitching config
  + source uri) and needs NO control-plane DB. It reuses orchestration._stitch_reel (shared core).
- Encoder: all clips in one reel must share ONE codec so the concat stays -c copy. NVENC is probed
  once and falls back to libx264 if unavailable (CI/non-GPU) ? the fallback must be all-or-nothing,
  never per-clip (which would break the concat).
- The config guard rejects validation=cloud_run_l4 without a cloud segment (nobody-validates footgun).
- Marker prefixes: compile uses _compile/, detect uses _dispatch/ ? must not collide.

Only report DEFECTS in the changed code (logic bugs, broken invariants, edge cases that produce a
wrong result / crash / silent data loss, or missing test coverage of a real failure mode). Do NOT
report style, naming, or speculative concerns. Cite exact file:line from undefined.

A reviewer claims this DEFECT in the merged #521 code. Try HARD to REFUTE it by reading the actual code at undefined. Default to is_real=false if you cannot concretely confirm the failure. Skeptic #1.

CLAIM (high/test-coverage) backend/orchestration.py:1263

summary: No test exercises the L4 worker's real _stitch_reel mirror-upload; a swallowed mirror failure there reports success (rc 0) but the reel is lost, so the SPA gets a 404 for a job it was told succeeded.
failure scenario: On the L4, a compile job runs cmd_compile -> orchestration._stitch_reel. The ffmpeg stitch succeeds locally, then `storage.put(out_path, gs://.../highlights/<vid>/<name>)` fails (transient GCS 5xx, throttling, or a creds/permission hiccup). The `except Exception` at orchestration.py:1263 swallows it (logs a warning) and _stitch_reel returns `status="success"` with the intended `reel_uri` (line 1270-1276). cmd_compile (worker/__main__.py:513-514) sets rc=0 purely from that status and writes a SUCCESS done-marker; _dispatch_compile_remote returns {status: success, download_url}. But the reel was never uploaded and the worker's local file is discarded when the Job exits, so /api/highlights -> signed_reel_url -> storage.exists() is False, the control-plane has no local fallback (the stitch ran on the worker), and the download 404s forever. Every #521 test hides this: test_worker.py's cmd_compile tests monkeypatch orchestration._stitch_reel, and test_compile_dispatch.py / test_compute_backend.py use a fake CloudRunJobClient that writes the marker directly, so the real worker mirror path (and its swallow-on-failure) is never run.

2. assistant (2026-07-08T06:14:20.175Z)

I'll investigate this claim by reading the actual code. Let me start with the key files.

3. user (2026-07-08T06:14:21.197Z)

```
1180    def signed_reel_url(cfg, video_id: str, name: str) -> Optional[str]:
1181        """A short-lived signed URL for a reel mirrored to the output bucket, so
/api/highlights can
1182        307-redirect off the app's response path (bypassing Cloud Run's ~32 MiB cap + the
ephemeral local
1183        /tmp; #463). None ? no bucket copy ? serve the local file. Uses the active storage
config for BOTH
1184        the existence check and the signing (extracted here to keep backend/main.py lean ?
P23/P24)."""
1185        reel_uri = reel_object_uri(cfg, video_id, name)
1186        if not reel_uri:
1187            return None
1188        settings = _storage_settings(cfg)
1189        if storage.exists(reel_uri, config=settings):
1190            return storage.signed_url(reel_uri, config=settings, method="GET")
1191        return None
1192
1193
1194    def _compile_wait_message(elapsed_sec: float) -> str:
1195        """The live job.progress line while an L4 compile/stitch is in flight (#521; mirrors
1196        ``_wait_progress_message``). Minute-granular; under a minute reads as cold-start."""
1197        minutes = int(elapsed_sec) // 60
1198        if minutes < 1:
1199            return "stitching on cloud GPU (NVENC) ? starting up"
1200        return f"stitching on cloud GPU (NVENC) ? {minutes} min elapsed"
1201
1202
1203    def _stitch_reel(
1204        cfg,
1205        stitching,
1206        video_id: str,
1207        source_ref: str,
1208        time_pairs: List[Tuple[float, float]],
1209        out_name: str,
1210        locale: Optional[str],
1211        notify,
1212    ) -> Dict[str, Any]:
1213        """Stitch + mirror-upload core, shared by the control-plane compile handler AND the
L4 worker
1214        (#521). ``time_pairs`` are already RESOLVED (the caller filtered via
``_resolve_compile``), so this
1215        needs NO control-plane DB ? the worker reads them from the staged spec. Builds the
```

```
VideoCompiler
        with ``stitching.encoder`` (NVENC on the L4, libx264 on the CP), fetches the source,
runs the
        trim/watermark/concat, mirrors the reel to
``<output_root>/highlights/<video_id>/<name>``, and
        returns the ``{status, filepath, download_url, video_id, reel_uri}`` the SPA polls
for. Returns a
        ``status='failed'`` result on a stitch error (never raises for an expected ffmpeg
failure)."""
        import backend.main as _main  # call-time seam: VideoCompiler stays patchable on
backend.main

    <<<<<<< Updated upstream
    =======
        notify = progress or (lambda stage: None)
        params = json.loads(job.get("params") or "{}")
        video_id = job.get("video_id") or ""
        segment_ids = params.get("segment_ids") or []
        locale = params.get("locale")
        rendition = str(params.get("rendition") or "original")
        if rendition not in {"original", "sd"}:
            return {
                "status": "failed",
                "message": f"Unsupported compile rendition: {rendition}",
                "video_id": video_id,
            }

        notify("resolving")
        try:
            video, kept, out_name = _resolve_compile(
                db, cfg, video_id, segment_ids, params.get("output_filename", "")
            )
        except _CompileError as e:
            return {"status": "failed", "message": e.detail, "video_id": video_id}

        stitching = getattr(cfg, "stitching", None) or StitchingConfig()
    >>>>>>> Stashed changes
        output_dir = (
            getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
        )
        # Localize the branding watermark to the UI locale the SPA rendered in (recorded on
the job);
        # unknown/absent locale keeps the configured default unchanged.
        localized_watermark = localize_watermark(stitching.watermark, locale)
        encode_profile = _rendition_encode_profile(stitching, rendition)
        compiler = _main.VideoCompiler(
            output_dir,
            begin_padding=stitching.begin_padding,
            end_padding=stitching.end_padding,
            watermark=localized_watermark,
    <<<<<<< Updated upstream
            encoder=getattr(stitching, "encoder", "libx264"),
    =======
            encode_profile=encode_profile,
    >>>>>>> Stashed changes
        )

        notify("stitching")
        logger.info(
            "[compile] stitching video_id=%s rendition=%s output=%s",
            video_id,
            rendition,
            out_name,
        )
        local_src = ""
```

```
1274        try:
1275            # The stored `filepath` is the SOURCE ref (gs://, s3://, gdrive://, or a local
path). Resolve to
1276            # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-
scratch for a URI (the
1277            # same seam the process path uses); without it, compiling a bucket-ingested
video can't open gs://.
1278            local_src = storage.acquire_local_input(
1279                source_ref, getattr(cfg, "storage", None)
1280            )
1281            # The ffmpeg stitch is CPU-heavy ? serialize it so a raised drain worker count
(for remote
1282            # dispatch) doesn't run multiple compiles at once on the box (see
_LOCAL_COMPUTE_LANE).
1283            with _LOCAL_COMPUTE_LANE:
1284                out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
1285        except Exception as e:  # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job
FAILURE, not a crash
1286            logger.error(
1287                "[compile] stitching failed for video_id=%s: %s", video_id, e, exc_info=True
1288            )
1289            return {
1290                "status": "failed",
1291                "message": f"Highlight stitching failed: {e}",
1292                "video_id": video_id,
1293            }
1294        finally:
1295            if local_src:
1296                storage.cleanup_acquired_local_input(
1297                    source_ref, local_src, getattr(cfg, "storage", None)
1298                )
1299
1300        # #463: mirror the reel to the output bucket so /api/highlights can 307 to a signed
URL ? Cloud Run
1301        # caps a buffered response at ~32 MiB (a real reel is bigger) and the local /tmp
copy dies when the
1302        # instance recycles. A mirror-upload failure must NOT fail an otherwise-good stitch
(the endpoint
1303        # still falls back to the local file on the same instance). On the L4 worker the
mirror is how the
1304        # reel reaches the bucket at all (the worker's local file is discarded when the Job
exits).
1305        reel_uri = reel_object_uri(cfg, video_id, out_name)
1306        if reel_uri:
1307            try:
1308                notify("uploading reel")
1309                # Thread the active storage settings so the configured backend (S3
endpoint/region, GCS
1310                # project/client) is used, not a fresh default.
1311                storage.put(out_path, reel_uri, config=_storage_settings(cfg))
1312            except Exception as e:  # noqa: BLE001 ? best-effort mirror; local stitch
already succeeded
1313                logger.warning("[compile] could not mirror reel to %s: %s", reel_uri, e)
1314
1315        # `filepath` is server-local (not browser-reachable); also return the download URL
so the SPA needn't
1316        # hand-build it (P6 / PER_RALLY_SELECTION.md §4.3).
1317        download_url = f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
1318        notify("finished")
1319        return {
1320            "status": "success",
1321            "filepath": out_path,
1322            "download_url": download_url,
1323            "video_id": video_id,
1324    <<<<<<< Updated upstream
```

```
1325            "reel_uri": reel_uri or "",
1326    =======
1327            "filename": out_name,
1328            "rendition": rendition,
1329    >>>>>>> Stashed changes
1330        }
1331
1332
1333    def _dispatch_compile_remote(
1334        cfg,
1335        db,
1336        video_id: str,
1337        segment_ids: List[int],
1338        output_filename: str,
1339        locale: Optional[str],
```

4. user (2026-07-08T06:14:22.795Z)

```
dac6736 perf(serving): run compile/stitch on the L4 worker; config-driven phase?machine
placement (#521) (#525)
bf97700 docs(runlogs): record live SPA CUJ closing the 2 owner-gated validation-latency cells
(§9) (#522)
699d85b feat(validation): cache validation verdict by video_id to skip re-validation on reupload
(#518)
71c88ef docs(runlogs): validation-latency closure report for #512 + #513 (cloud-serving) (#519)
115b83d docs(runlogs): record live smoke of the crash-before-marker fast-fail (#515) (#517)
ef41473 fix(cloud-run): fail fast when a worker execution terminates without a done-marker
(#515)
f5c9dd9 chore(ui): re-vendor SPA bundle with the hosted compute-picker cloud default (#516)
ec2d080 fix(serving): fail fast when compute=local forces weightless in-process detection (#511)
```

5. assistant (2026-07-08T06:14:31.870Z)

The working tree has merge conflict markers ? this isn't the clean merged #521 code. Let me read the file at the actual merge commit `dac6736`.

6. user (2026-07-08T06:14:34.601Z)

```
M  README.md
M  SESSION_HANDOFF.md
M  backend/config/models.py
M  backend/infrastructure/database_jobs.py
M  backend/main.py
UU backend/orchestration.py
UU backend/pipeline/stitcher/compiler.py
M  config.json
M  tests/test_api_endpoints.py
M  tests/test_async_jobs.py
M  tests/test_compiler.py
M  tests/test_watermark.py
?? score_golden_gate.py
?? score_local.py
---STASH---
stash@{0}: On codex/issue-523-sd-reel-plan: sd-reel-engine-wip
```

7. user (2026-07-08T06:14:37.977Z)

```
1460 C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-khelsutra-guru-badminton-
highlight-indexer\75d3d121-f240-4293-afa5-b436a379ad31\scratchpad\orchestration_merged.py
```

8. assistant (2026-07-08T06:14:44.100Z)

The working tree is mid-conflict from a different branch. I'll review the actual merged #521 code from commit `dac6736`. Let me read the clean extracted version.

9. user (2026-07-08T06:14:45.163Z)

```
1200        """Stitch + mirror-upload core, shared by the control-plane compile handler AND the
L4 worker
1201        (#521). ``time_pairs`` are already RESOLVED (the caller filtered via
``_resolve_compile``), so this
1202        needs NO control-plane DB ? the worker reads them from the staged spec. Builds the
VideoCompiler
1203        with ``stitching.encoder`` (NVENC on the L4, libx264 on the CP), fetches the source,
runs the
1204        trim/watermark/concat, mirrors the reel to
``<output_root>/highlights/<video_id>/<name>``, and
1205        returns the ``{status, filepath, download_url, video_id, reel_uri}`` the SPA polls
for. Returns a
1206        ``status='failed'`` result on a stitch error (never raises for an expected ffmpeg
failure)."""
1207        import backend.main as _main  # call-time seam: VideoCompiler stays patchable on
backend.main
1208
1209        output_dir = (
1210            getattr(getattr(cfg, "output", None), "output_dir", "output") or "output"
1211        )
1212        # Localize the branding watermark to the UI locale the SPA rendered in (recorded on
the job);
1213        # unknown/absent locale keeps the configured default unchanged.
1214        localized_watermark = localize_watermark(stitching.watermark, locale)
1215        compiler = _main.VideoCompiler(
1216            output_dir,
1217            begin_padding=stitching.begin_padding,
1218            end_padding=stitching.end_padding,
1219            watermark=localized_watermark,
1220            encoder=getattr(stitching, "encoder", "libx264"),
1221        )
1222
1223        notify("stitching")
1224        local_src = ""
1225        try:
1226            # The stored `filepath` is the SOURCE ref (gs://, s3://, gdrive://, or a local
path). Resolve to
1227            # a LOCAL file the stitcher can read ? identity for a local path, fetch-to-
scratch for a URI (the
1228            # same seam the process path uses); without it, compiling a bucket-ingested
video can't open gs://.
1229            local_src = storage.acquire_local_input(
1230                source_ref, getattr(cfg, "storage", None)
1231            )
1232            # The ffmpeg stitch is CPU-heavy ? serialize it so a raised drain worker count
(for remote
1233            # dispatch) doesn't run multiple compiles at once on the box (see
_LOCAL_COMPUTE_LANE).
1234            with _LOCAL_COMPUTE_LANE:
1235                out_path = compiler.compile_highlights(local_src, time_pairs, out_name)
1236        except Exception as e:  # noqa: BLE001 ? a stitch/ffmpeg failure is a clean job
FAILURE, not a crash
1237            logger.error(
1238                "[compile] stitching failed for video_id=%s: %s", video_id, e, exc_info=True
1239            )
1240            return {
1241                "status": "failed",
1242                "message": f"Highlight stitching failed: {e}",
1243                "video_id": video_id,
1244            }
1245        finally:
1246            if local_src:
1247                storage.cleanup_acquired_local_input(
```

```
                    source_ref, local_src, getattr(cfg, "storage", None)
                )

        # #463: mirror the reel to the output bucket so /api/highlights can 307 to a signed
URL ? Cloud Run
        # caps a buffered response at ~32 MiB (a real reel is bigger) and the local /tmp
copy dies when the
        # instance recycles. A mirror-upload failure must NOT fail an otherwise-good stitch
(the endpoint
        # still falls back to the local file on the same instance). On the L4 worker the
mirror is how the
        # reel reaches the bucket at all (the worker's local file is discarded when the Job
exits).
        reel_uri = reel_object_uri(cfg, video_id, out_name)
        if reel_uri:
            try:
                notify("uploading reel")
                # Thread the active storage settings so the configured backend (S3
endpoint/region, GCS
                # project/client) is used, not a fresh default.
                storage.put(out_path, reel_uri, config=_storage_settings(cfg))
            except Exception as e:  # noqa: BLE001 ? best-effort mirror; local stitch
already succeeded
                logger.warning("[compile] could not mirror reel to %s: %s", reel_uri, e)

        # `filepath` is server-local (not browser-reachable); also return the download URL
so the SPA needn't
        # hand-build it (P6 / PER_RALLY_SELECTION.md §4.3).
        download_url = f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
        notify("finished")
        return {
            "status": "success",
            "filepath": out_path,
            "download_url": download_url,
            "video_id": video_id,
            "reel_uri": reel_uri or "",
        }


    def _dispatch_compile_remote(
        cfg,
        db,
        video_id: str,
        segment_ids: List[int],
        output_filename: str,
        locale: Optional[str],
        notify,
    ) -> Dict[str, Any]:
        """#521: run the compile/stitch on the L4 worker instead of the CPU-only control-
plane. The
        control-plane owns the DB, so it RESOLVES the reel's segments here
(``_resolve_compile``), stages a
        small compile-spec (resolved time-pairs + the resolved stitching/watermark config +
the source URI)
        to the bucket, dispatches a ``compile`` Cloud Run Job on the L4 (NVENC), bucket-
polls the
        done-marker, and returns the same ``{status, filepath, download_url}`` shape the SPA
expects. The
        worker needs NO control-plane DB ? everything it needs rides the spec. Mirrors
        ``_maybe_dispatch_remote`` (the DETECT dispatch)."""
        from backend.compute import CompileJobSpec, get_compute_backend

        notify("resolving")
        try:
            video, kept, out_name = _resolve_compile(
```

```
                db, cfg, video_id, segment_ids, output_filename
            )
        except _CompileError as e:
            return {"status": "failed", "message": e.detail, "video_id": video_id}

        stitching = getattr(cfg, "stitching", None) or StitchingConfig()
        storage_cfg = getattr(cfg, "storage", None)
        out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
        compute = get_compute_backend(cfg, target_override="cloud_run") if out_root else
None
        if compute is None:
            # Compile was placed on the L4 but the cloud dispatch isn't actually wired here
(no output
            # bucket / Cloud Run env). Fall back to the in-process stitch rather than fail ?
a slow reel
            # beats no reel. (The placement guard means this is only reachable in a mis-
provisioned setup.)
            logger.warning(
                "[compile] compile placed on the L4 but no cloud backend available
(out_root=%r) ? "
                "stitching in-process on the control-plane instead.",
                out_root,
            )
            time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
            return _stitch_reel(
                cfg, stitching, video_id, video["filepath"], time_pairs, out_name, locale,
notify
            )

        # Build the compile-spec the worker consumes. time-pairs are JSON lists (frozen-safe
on the wire);
        # the stitching config is the control-plane's RESOLVED one, with the encoder swapped
to the L4's
        # (deployment.cloud_stitch_encoder, NVENC by default) so the worker hardware-encodes
the reel.
        stitch_dump = stitching.model_dump()
        stitch_dump["encoder"] = getattr(
            getattr(cfg, "deployment", None), "cloud_stitch_encoder", "h264_nvenc"
        )
        spec_payload = {
            "video_id": video_id,
            "video_uri": video["filepath"],
            "output_name": out_name,
            "locale": locale,
            "time_pairs": [
                [float(s["start_time"]), float(s["end_time"])] for s in kept
            ],
            "stitching": stitch_dump,
        }
        token = uuid.uuid4().hex
        spec_uri = f"{out_root.rstrip('/')}/_compile/{token}/spec.json"
        try:
            notify("staging compile spec")
            with tempfile.TemporaryDirectory(prefix="cr-compile-") as td:
                local_spec = os.path.join(td, "spec.json")
                with open(local_spec, "w", encoding="utf-8") as f:
                    json.dump(spec_payload, f)
                storage.put(local_spec, spec_uri, config=_storage_settings(cfg))
        except Exception as e:  # noqa: BLE001 ? a staging failure is a clean job failure,
not a crash
            logger.error(
                "[compile] failed to stage the compile spec to %s: %s", spec_uri, e,
exc_info=True
            )
            return {
```

```
1354                "status": "failed",
1355                "message": f"cloud compile: could not stage the compile spec: {e}",
1356                "video_id": video_id,
1357            }
1358
1359        # Forward the bucket coordinates so the worker (no DEPLOYMENT_PROFILE) can read the
spec, mirror the
```

10. assistant (2026-07-08T06:15:08.444Z)

Let me read the worker's `cmd_compile` and the rest of `_dispatch_compile_remote` from the merged commit.

11. user (2026-07-08T06:15:13.648Z)

```
  977 C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-khelsutra-guru-badminton-
highlight-indexer\75d3d121-f240-4293-afa5-b436a379ad31\scratchpad/worker_main_merged.py
  516 C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-khelsutra-guru-badminton-
highlight-indexer\75d3d121-f240-4293-afa5-b436a379ad31\scratchpad/cloud_run_merged.py
 1493 total
```

12. user (2026-07-08T06:15:14.072Z)

```
1359        # Forward the bucket coordinates so the worker (no DEPLOYMENT_PROFILE) can read the
spec, mirror the
1360        # reel, and write the marker ? mirrors how _maybe_dispatch_remote forwards the WASB
knobs for detect.
1361        overrides = [f"storage.output_root={out_root}"]
1362        backend_name = (getattr(storage_cfg, "backend", "") or "").strip()
1363        if backend_name:
1364            overrides.append(f"storage.backend={backend_name}")
1365        spec = CompileJobSpec(
1366            out_uri=out_root,
1367            spec_uri=spec_uri,
1368            video_id=video_id,
1369            output_name=out_name,
1370            config_overrides=tuple(overrides),
1371        )
1372
1373        notify("dispatching stitch (cloud_run)")
1374        logger.info(
1375            "[compile] dispatching stitch for video_id=%s -> Cloud Run Job (out=%s, %d
clip(s))",
1376            video_id,
1377            out_root,
1378            len(kept),
1379        )
1380        try:
1381            result = compute.run_compile(
1382                spec, on_wait=lambda elapsed: notify(_compile_wait_message(elapsed))
1383            )
1384        except Exception as e:  # noqa: BLE001 ? any dispatch/poll error is a clean job
failure, not a 500
1385            logger.error("[compile] cloud compile dispatch error: %s", e, exc_info=True)
1386            return {
1387                "status": "failed",
1388                "message": f"cloud compile dispatch error: {e}",
1389                "video_id": video_id,
1390            }
1391        if result.returncode != 0:
1392            msg = str(
1393                result.report.get("message")
1394                or f"cloud compile job failed (returncode {result.returncode})."
1395            )
```

```
            return {"status": "failed", "message": msg, "video_id": video_id}

        download_url = str(
            result.report.get("download_url")
            or f"/api/highlights/{quote(video_id)}/{quote(out_name)}"
        )
        logger.info(
            "[COMPILE] cloud_run stitched: execution=%s out=%s video_id=%s",
            result.execution,
            result.outputs_uri,
            video_id,
        )
        notify("finished")
        return {
            "status": "success",
            "filepath": str(result.report.get("reel_uri") or result.outputs_uri or ""),
            "download_url": download_url,
            "video_id": video_id,
            "compute": {
                "path": "cloud_run",
                "execution": result.execution,
                "job": os.environ.get("CLOUD_RUN_JOB", "rally-worker"),
                "region": os.environ.get("CLOUD_RUN_REGION", "asia-south1"),
            },
        }


    def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
        """Handler for a ``kind='compile'`` job (#329): re-resolve the segments, stitch the
reel, and return
        the ``{status, filepath, download_url}`` result the SPA polls for. Returns a
``status='failed'``
        result for an EXPECTED problem (missing video / no segments / a stitch error) so the
SPA always gets
        a clean verdict instead of a worker-crash error.

        #521 phase_placement.compile: when compile is placed on the L4 (``cloud_run_l4``)
and cloud
        dispatch is available, the CPU-bound 4K stitch is OFFLOADED to the GPU/NVENC worker
        (``_dispatch_compile_remote``) ? the 2026-07-08 CUJ measured ~13 min for an 11-clip
reel on the
        2-vCPU control-plane. DEFAULT (compile on ``control_plane``) stitches in-process
here, unchanged."""
        notify = progress or (lambda stage: None)
        params = json.loads(job.get("params") or "{}")
        video_id = job.get("video_id") or ""
        segment_ids = params.get("segment_ids") or []
        locale = params.get("locale")
        output_filename = params.get("output_filename", "")

        if (
            resolve_phase_placement(cfg, PHASE_COMPILE) == PLACE_CLOUD_L4
            and _cloud_available(cfg)
        ):
            return _dispatch_compile_remote(
                cfg, db, video_id, segment_ids, output_filename, locale, notify
            )

        notify("resolving")
        try:
            video, kept, out_name = _resolve_compile(
                db, cfg, video_id, segment_ids, output_filename
            )
        except _CompileError as e:
            return {"status": "failed", "message": e.detail, "video_id": video_id}
```

```
1455
1456        stitching = getattr(cfg, "stitching", None) or StitchingConfig()
1457        time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
1458        return _stitch_reel(
1459            cfg, stitching, video_id, video["filepath"], time_pairs, out_name, locale,
notify
```

13. assistant (2026-07-08T06:15:19.595Z)

Now let me read the worker's `cmd_compile` handler.

14. user (2026-07-08T06:15:22.382Z)

```
400             status,
401             storage_cfg,
402             str(getattr(args, "video_uri", "") or ""),
403         )
404
405         print(
406             f"[worker] infer done: {len(segments)} segment(s); pushed {len(pushed)}
artifact(s):"
407         )
408         for u in pushed:
409             print("   ", u)
410
411         # M6: a remote dispatcher passes --done-uri; write the SUCCESS completion marker so
its bucket-poll
412         # terminates (carries video_id ? the dispatcher's output path + rc=0). Worker
FAILURES (rc 2/3)
413         # write their own marker via _fail() above, so the dispatcher fails FAST either way
(no 30-min hang
414         # on a bad video). DEFAULT-OFF: no --done-uri ? no marker written (the unchanged
local run).
415         if getattr(args, "done_uri", None):
416             _write_done_marker(
417                 args.done_uri, video_id, 0, len(segments), status, storage_cfg, scratch
418             )
419         return 0
420
421
422     def _read_compile_spec(spec_uri: str, storage_cfg: dict, scratch: str) -> dict:
423         """Fetch + parse the control-plane-staged compile-spec (#521): the resolved
``time_pairs``, the
424         resolved ``stitching`` config, the source ``video_uri``, ``video_id``,
``output_name`` + ``locale``.
425         A gs://|s3:// URI is downloaded to scratch first (cloud backends require a real
dst)."""
426         local = storage.fetch(
427             spec_uri, os.path.join(scratch, "compile_spec.json"), config=storage_cfg
428         )
429         with open(local, encoding="utf-8") as f:
430             data = json.load(f)
431         if not isinstance(data, dict):
432             raise ValueError(f"compile spec at {spec_uri} is not a JSON object")
433         return data
434
435
436     def _write_compile_marker(
437         done_uri: str,
438         result: dict,
439         video_id: str,
440         returncode: int,
441         storage_cfg: dict,
442         scratch: str,
443     ) -> None:
```

```
        """#521: write the compile done-marker (rc + status + the reel's
download_url/reel_uri) so the
        dispatcher's bucket-poll terminates and can return the SPA result. Best-effort ?
never raises."""
        try:
            marker = {
                "video_id": video_id,
                "returncode": returncode,
                "status": result.get("status", "failed"),
                "download_url": result.get("download_url", ""),
                "reel_uri": result.get("reel_uri", ""),
                "message": result.get("message", ""),
            }
            local = os.path.join(scratch, "_compile_done.json")
            with open(local, "w", encoding="utf-8") as f:
                json.dump(marker, f)
            storage.put(local, done_uri, config=storage_cfg)
            logger.info("[worker] wrote compile marker -> %s", done_uri)
        except Exception as e:  # never fail a good stitch because the marker push hiccuped
            logger.warning("[worker] could not write compile done-marker %s: %s", done_uri,
e)


    def cmd_compile(args: argparse.Namespace) -> int:
        """#521: run the reel COMPILE/stitch on THIS worker (the L4 has NVENC + 8 vCPU). The
stitch is the
        2026-07-08 CUJ's dominant term (~13 min on the 2-vCPU control-plane for an 11-clip
4K reel). Read
        the control-plane-staged compile-spec (resolved time-pairs + stitching config +
source URI), stitch
        the reel via the SAME ``orchestration._stitch_reel`` core the control-plane uses (so
there is no
        forked stitch pipeline), upload the reel to the bucket, and write a done-marker for
the dispatcher's
        bucket-poll. No control-plane DB is needed ? the spec carries everything the stitch
requires."""
        from backend import orchestration
        from backend.config import StitchingConfig

        config = load_config(args.config) if args.config else load_config()
        _apply_overrides(config, args.set)
        storage_cfg = config.storage.model_dump()

        scratch = args.scratch or tempfile.mkdtemp(prefix="rally-compile-")
        os.makedirs(scratch, exist_ok=True)
        # Clips + the assembled reel land in scratch; the reel is then mirror-uploaded to
the bucket.
        config.output.output_dir = scratch
        # The dispatcher forwards storage.output_root via --set; --out-uri carries the same
value as a
        # fallback so reel_object_uri (which reads storage.output_root) always resolves the
reel's bucket
        # destination even if the override were dropped.
        if not (config.storage.output_root or "").strip():
            config.storage.output_root = args.out_uri

        spec = _read_compile_spec(args.spec_uri, storage_cfg, scratch)
        video_id = str(spec.get("video_id") or "")
        out_name = str(spec.get("output_name") or "")
        source_ref = str(spec.get("video_uri") or "")
        locale = spec.get("locale")
        time_pairs = [
            (float(pair[0]), float(pair[1])) for pair in (spec.get("time_pairs") or [])
        ]
        stitching = StitchingConfig(**(spec.get("stitching") or {}))
```

```
497
498         print(
499             f"[worker] compile: video_id={video_id} clips={len(time_pairs)} "
500             f"encoder={stitching.encoder} out={out_name}"
501         )
502
503         result = orchestration._stitch_reel(
504             config,
505             stitching,
506             video_id,
507             source_ref,
508             time_pairs,
509             out_name,
510             locale,
511             lambda stage: print(f"[worker] compile: {stage}"),
512         )
513         status = str(result.get("status") or "failed")
514         rc = 0 if status == "success" else 3
515         if status != "success":
516             print(f"[worker] compile FAILED: {result.get('message')}")
517
518         # DEFAULT-OFF: no --done-uri ? no marker (a direct-CLI stitch). A remote dispatcher
passes it.
519         if getattr(args, "done_uri", None):
520             _write_compile_marker(args.done_uri, result, video_id, rc, storage_cfg, scratch)
521         return rc
522
523
524     #: Video extensions accepted under <corpus>/videos/ (mirrors the labels/ '.csv' filter
so a
525     #: per-video sidecar ? .srt/.json/thumbnail ? can't shadow the real video on a stem
collision).
526     _VIDEO_EXTS = (".mp4", ".mov", ".mkv", ".avi", ".webm", ".m4v")
527
528     #: Leading ISO-date prefix on a canonical label name (``labels/<YYYY-MM-
DD>_<name>.rallies.csv``,
529     #: docs/DATA_IN_GCS.md §7.1). It disambiguates recycled GoPro names by authoring date
but is NOT
530     #: part of the clip's join key, so it's stripped when deriving the videos/ stem.
531     _LABEL_DATE_PREFIX = re.compile(r"^\d{4}-\d{2}-\d{2}_")
532
533
534     def _label_clip_name(fname: str) -> str:
535         """Derive the clip name (the ``videos/<name>.<ext>`` join key) from a label
filename.
536
537         Strips both the ``.rallies.csv`` / ``.csv`` suffix and an optional leading ISO-date
prefix, so a
538         canonical dated label and a legacy bare label resolve to the SAME bare clip stem:
539
540         * ``2026-06-22_GX020094.rallies.csv`` -> ``GX020094``       (canonical, date-
stamped)
541         * ``GX020094.rallies.csv``            -> ``GX020094``       (legacy bare ? same
stem)
542         * ``GX020094_proxy.rallies.csv``      -> ``GX020094_proxy`` (``_proxy`` is part of
the name, kept)
543         * ``2026-06-21_mahadevpura_1.rallies.csv`` -> ``mahadevpura_1`` (underscores in
<name> survive)
544
545         Without the date strip, ``--trajectory-source detector`` can't match a dated label
to its bare
546         video stem, so every clip is skipped and train aborts with ``<2 videos``
(DATA_IN_GCS §7b #1).
547         """
548         name = fname.replace(".rallies.csv", "").replace(".csv", "")
```

```
549         return _LABEL_DATE_PREFIX.sub("", name)
550
551
552     def _probe_video_fps_width(path: str):
553         """Robust ffprobe (fps, frame_width) for a real video. Returns None on ANY failure.
554
555         The detector trajectory is frame-indexed; the harness maps frame->time via fps, so a
WRONG
556         fps silently mis-aligns every rally label (e.g. a 60fps clip read as 30 doubles
every time).
557         cv2's ``_probe_fps_width`` silently defaults to (30, 1280) on a read miss ? fine for
the
558         inference report (it flags the fallback) but corrupting for REAL training data. So
here we
559         probe with ffprobe (same tool as ``get_video_duration``) and return None so the
caller SKIPS
560         rather than guesses.
561         """
562         import json
563         import subprocess
564
565         try:
566             out = subprocess.check_output(
567                 [
568                     ffprobe_bin(),
569                     "-v",
570                     "error",
571                     "-select_streams",
572                     "v:0",
573                     "-show_entries",
574                     "stream=r_frame_rate,width",
575                     "-of",
576                     "json",
577                     path,
578                 ],
579                 stderr=subprocess.DEVNULL,
```

15. assistant (2026-07-08T06:16:22.509Z)

The worker's `cmd_compile` sets `rc = 0` purely from `_stitch_reel`'s returned `status`, with no `storage.exists` verification of the uploaded reel. Now let me verify the remaining links in the claim: the tests, and the `/api/highlights` fallback behavior.

16. user (2026-07-08T06:16:23.955Z)

```
246:    signed_reel_url,
1045:@app.get("/api/highlights/{video_id}/{filename}")
1046:def download_highlight(video_id: str, filename: str, request: Request):
1058:    # exists/sign logic (with the active storage config) lives in
orchestration.signed_reel_url to keep
1061:        signed = signed_reel_url(request.app.state.config, video_id, name)
```

17. user (2026-07-08T06:16:28.308Z)

```
test_api_cloud_dispatch.py
test_compile_dispatch.py
test_compiler.py
test_compute_backend.py
test_job_worker_loop.py
test_job_worker_module_seam.py
test_worker.py
```

18. user (2026-07-08T06:16:31.347Z)

```
1045    @app.get("/api/highlights/{video_id}/{filename}")
1046    def download_highlight(video_id: str, filename: str, request: Request):
1047        """Stream a compiled highlight reel ? `filename` is the bare name given to
/api/compile
1048        (which only writes into output_dir; the browser previously got back an unreachable
1049        server-local path)."""
1050        try:
1051            name = safe_output_filename(filename)
1052        except ValueError as e:
1053            raise HTTPException(status_code=400, detail=str(e)) from e
1054        if not request.app.state.db.get_video(video_id):
1055            raise HTTPException(status_code=404, detail="Indexed video record not found.")
1056        # #463: when the reel was mirrored to the output bucket, 307 to a short-lived signed
URL (bytes flow
1057        # client<->store, bypassing Cloud Run's ~32 MiB response cap + the ephemeral local
/tmp). The
1058        # exists/sign logic (with the active storage config) lives in
orchestration.signed_reel_url to keep
1059        # this module lean; best-effort ? any error falls back to the local file below.
1060        try:
1061            signed = signed_reel_url(request.app.state.config, video_id, name)
1062            if signed:
1063                return RedirectResponse(signed, status_code=307)
1064        except Exception as e:  # noqa: BLE001 ? signing/exists hiccup ? serve the local
file instead
1065            logger.warning("[highlights] signed-url path failed for %s; serving local: %s",
video_id, e)
1066        output_dir = (
1067            getattr(
1068                getattr(request.app.state.config, "output", None), "output_dir", "output"
1069            )
1070            or "output"
1071        )
1072        path = os.path.join(output_dir, name)
1073        try:
1074            path = resolve_under_root(path, output_dir)
1075        except ValueError as e:
1076            raise HTTPException(status_code=400, detail=str(e)) from e
1077        if not os.path.isfile(path):
1078            raise HTTPException(
1079                status_code=404,
1080                detail=f"No compiled file named '{name}'. Compile first via /api/compile.",
1081            )
1082        media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
1083        return FileResponse(path, media_type=media, filename=name)
1084
1085
1086    @app.post("/api/query")
1087    def query_play_segments(req: QueryRequest, request: Request):
1088        filters = {
1089            "server": req.server,
1090            "receiver": req.receiver,
1091            "duration_min": req.duration_min,
1092            "event_type": req.event_type,
1093        }
1094        segments = request.app.state.db.query_play_segments(req.video_id, filters)
1095        if not segments and read_path_rehydrate(
1096            request.app.state.db, request.app.state.config, req.video_id
1097        ):  # #485: DB = cache, bucket = truth ? re-hydrate before reporting nothing
1098            segments = request.app.state.db.query_play_segments(req.video_id, filters)
1099        return {"play_segments": segments}
```

19. assistant (2026-07-08T06:16:41.602Z)

The endpoint confirms: mirror-failed ? `signed` is None ? falls back to local file at

`output_dir/name`, which doesn't exist on the control-plane (stitch ran on the worker) ? 404.
Now let me verify the "no test" claim by examining the test files.

20. user (2026-07-08T06:16:45.991Z)

```
===== test_worker.py : _stitch_reel / mirror / storage.put refs =====
158:def mocked_pipeline(monkeypatch):
159:    monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vid123")
160:    monkeypatch.setattr(
165:    monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _FakeSeg)
166:    monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
236:def test_cmd_infer_zero_segments_is_logged_loudly(tmp_path, monkeypatch, caplog):
238:    monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidZ")
239:    monkeypatch.setattr(
244:    monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _EmptySeg)
245:    monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
246:    monkeypatch.setenv("RALLY_DETECTOR_IMPL", "native")
280:    tmp_path, monkeypatch
285:    monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidX")
286:    monkeypatch.setattr(
291:    monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _FailSeg)
292:    monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
325:def test_cmd_infer_real_trajectory_segmenter_failure_flows_to_report(tmp_path, monkeypatch):
338:    monkeypatch.setattr(WasbHybridSegmenter, "_resolve_runner", lambda self, cfg: (runner,
{}))
339:    monkeypatch.setattr(
342:    monkeypatch.setattr(
345:    monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidR")
346:    monkeypatch.setattr(
349:    monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: WasbHybridSegmenter)
350:    monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
381:def test_cmd_infer_validation_failure_returns_2(tmp_path, monkeypatch):
382:    monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidF")
383:    monkeypatch.setattr(
388:    monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
419:def test_cmd_infer_skip_validation_bypasses_validators(tmp_path, monkeypatch):
425:    monkeypatch.setattr(wmain, "compute_video_id", lambda p: "vidSkip")
430:    monkeypatch.setattr(wmain.validator_registry, "run_all_with_context", _boom)
431:    monkeypatch.setattr(wmain.segmenter_registry, "get", lambda name: _FakeSeg)
432:    monkeypatch.setattr(wmain, "emit_indexer_report", lambda **k: None)
484:def test_cmd_train_detector_joins_dated_labels_to_bare_videos(tmp_path, monkeypatch):
492:    monkeypatch.setenv("RALLY_DETECTOR_IMPL", "stub")  # CPU stub runner (no GPU/WSL)
493:    monkeypatch.setattr(wmain, "_probe_video_fps_width", lambda p: (30.0, 1280.0))
525:    monkeypatch.setattr(subprocess, "run", fake_run)
555:def test_cmd_train_detector_skips_label_with_no_matching_video(tmp_path, monkeypatch):
561:    monkeypatch.setenv("RALLY_DETECTOR_IMPL", "stub")
562:    monkeypatch.setattr(wmain, "_probe_video_fps_width", lambda p: (30.0, 1280.0))
592:    monkeypatch.setattr(subprocess, "run", fake_run)
624:    tmp_path, monkeypatch
643:    monkeypatch.setattr(
648:    monkeypatch.setattr(
662:    monkeypatch.setattr(
681:    monkeypatch.setattr(subprocess, "run", fake_run)
708:    tmp_path, monkeypatch
715:    monkeypatch.setenv("RALLY_DETECTOR_IMPL", "stub")
716:    monkeypatch.setattr(wmain, "_probe_video_fps_width", lambda p: (30.0, 1280.0))
732:    monkeypatch.setattr(
758:def test_cmd_train_orchestration(tmp_path, monkeypatch):
785:    monkeypatch.setattr(subprocess, "run", fake_run)
831:def test_cmd_train_detector_source_runs_real_extract_via_stub(tmp_path, monkeypatch):
837:    monkeypatch.setenv(
841:    monkeypatch.setattr(wmain, "_probe_video_fps_width", lambda p: (30.0, 1280.0))
870:    monkeypatch.setattr(subprocess, "run", fake_run)
904:    tmp_path, monkeypatch
908:    monkeypatch.setenv("RALLY_DETECTOR_IMPL", "native")
```

```
989:def test_cmd_compile_stitches_and_writes_marker(tmp_path, monkeypatch):
990:    """cmd_compile reads the staged spec, runs the SHARED _stitch_reel core (VideoCompiler
stubbed so
991:    no ffmpeg), mirrors the reel, and writes a compile done-marker for the dispatcher's
bucket-poll."""
1015:    monkeypatch.setattr(orch, "_stitch_reel", _fake_stitch)
1019:    rc = wmain.cmd_compile(
1048:def test_cmd_compile_failure_returns_rc3_and_failed_marker(tmp_path, monkeypatch):
1060:    monkeypatch.setattr(
1062:        "_stitch_reel",
1067:    rc = wmain.cmd_compile(
1089:def test_cmd_compile_no_done_uri_writes_no_marker(tmp_path, monkeypatch):
1099:    monkeypatch.setattr(
1100:        orch, "_stitch_reel", lambda *a, **k: {"status": "success", "video_id": "vidReel"}
1104:    rc = wmain.cmd_compile(
```

21. user (2026-07-08T06:16:47.164Z)

```
===== test_compile_dispatch.py =====
4:(``deployment.phase_placement.compile=cloud_run_l4``), mirroring the DETECT dispatch ? all
GPU-free +
5:cloud-free via a FAKE ``CloudRunJobClient`` that simulates the compile worker (reads the
staged
17:from backend.compute.cloud_run import CloudRunJobClient
25:class _FakeCompileClient(CloudRunJobClient):
108:def test_compile_dispatches_to_l4_and_returns_reel(compile_env):
110:    fake = _FakeCompileClient()
123:def test_compile_dispatch_stages_resolved_spec_with_nvenc(compile_env):
128:    fake = _FakeCompileClient()
145:    assert "storage.output_root=" in joined  # bucket forwarded so the worker mirrors the
reel
148:def test_compile_dispatch_forwards_stitch_config_overrides(compile_env):
150:    (storage.output_root/backend) ? else the worker can't read the spec or mirror the
reel."""
152:    fake = _FakeCompileClient()
160:def test_compile_cloud_failure_returns_failed(compile_env):
164:    fake = _FakeCompileClient(returncode=3)
173:def test_compile_default_stitches_in_process_no_dispatch(compile_env, monkeypatch):
174:    """DEFAULT (compile on the control-plane): _execute_compile stitches in-process via
_stitch_reel
179:    fake = _FakeCompileClient()
188:    monkeypatch.setattr(orch, "_stitch_reel", _fake_stitch)
197:def test_compile_missing_video_fails_before_dispatch(compile_env):
200:    fake = _FakeCompileClient()
```

22. assistant (2026-07-08T06:16:56.303Z)

The `cmd_compile` tests all monkeypatch `orch._stitch_reel`, and the dispatch tests use a fake client. Let me confirm no test anywhere exercises the real `_stitch_reel` mirror path with a failing `storage.put`.

23. user (2026-07-08T06:17:05.445Z)

```
===== all _stitch_reel refs across tests =====
tests/test_compile_dispatch.py:206:    """DEFAULT (compile on the control-plane):
_execute_compile stitches in-process via _stitch_reel
tests/test_compile_dispatch.py:221:    monkeypatch.setattr(orch, "_stitch_reel", _fake_stitch)
tests/test_worker.py:990:    """cmd_compile reads the staged spec, runs the SHARED _stitch_reel
core (VideoCompiler stubbed so
tests/test_worker.py:1015:    monkeypatch.setattr(orch, "_stitch_reel", _fake_stitch)
tests/test_worker.py:1062:        "_stitch_reel",
tests/test_worker.py:1100:        orch, "_stitch_reel", lambda *a, **k: {"status": "success",
"video_id": "vidReel"}
```

```
===== tests that call real _stitch_reel (not setattr/monkeypatch) =====
```

24. user (2026-07-08T06:17:05.527Z)

```
===== tests referencing mirror failure / reel_uri exists / put failure in compile context =====
tests/test_api_assets.py:6:space (507); and put failure (502). Cloud SDKs are mocked; the Drive
path uses a real temp mount.
tests/test_api_assets.py:216:def test_put_failure_is_502(tmp_path, monkeypatch):
tests/test_api_cloud_dispatch.py:336:    Regression: the gate previously required input_backend
!= 'local' and hard-failed this shape."""
tests/test_api_cloud_dispatch.py:356:    assert out["status"] == "success"  # NOT the 'needs a
cloud input' pre-dispatch failure
tests/test_api_cloud_dispatch.py:367:def
test_cloud_local_upload_without_input_root_still_fails(cloud_env, tmp_path, monkeypatch):
tests/test_api_cloud_dispatch.py:368:    """No cloud input_root configured ? we genuinely can't
stage a local upload ? hard-fail (no
tests/test_api_cloud_dispatch.py:559:    """compute='cloud' on a server NOT wired for it fails
clearly (no silent local fallback, no
tests/test_api_cloud_dispatch.py:742:# them), so a forced compute='local' for
wasb/tracknet/fusion can't run in-process there. It must fail
tests/test_api_cloud_dispatch.py:800:    compute='local' on a cloud server is unaffected ? it
runs in-process, never fails fast."""
tests/test_api_cloud_dispatch.py:850:    (the default), compute='local' on a cloud_run server
fails fast, never dispatching."""
tests/test_api_cloud_dispatch.py:982:    """compute='cloud' on a server that can't dispatch
fails clearly AND records it: target cloud_run,
tests/test_api_endpoints.py:337:    Override output_dir; failed run must not pollute literal
'output/'; success uses configured.
tests/test_api_endpoints.py:781:    """A LOCAL input that doesn't exist still fast-fails 404 ?
the existence gate is preserved
tests/test_api_endpoints.py:862:def
test_process_remote_input_cleans_scratch_when_hash_fails(env):
tests/test_api_endpoints.py:878:    mp.setattr(m, "compute_video_id", fail_hash)
tests/test_input_source.py:184:def
test_acquire_local_input_cleans_scratch_when_fetch_fails(monkeypatch):
tests/test_restart_recovery.py:10:  (``outputs/<md5>/report.json``) for those rows ? completing
or failing them honestly,
tests/test_storage_gdrive_api.py:248:def test_put_failure_mid_upload_keeps_old_object_intact(be,
tmp_path):
tests/test_storage_gdrive_api.py:249:    """A re-put whose upload fails must leave the PREVIOUS
object readable. The old
tests/test_tracknet_hybrid.py:143:    """F1: an unreadable/<=0 video duration is a broken input
? a Failure, not a 0-rally []."""
Binary file tests/__pycache__/test_api_assets.cpython-313-pytest-9.0.3.pyc matches
Binary file tests/__pycache__/test_api_cloud_dispatch.cpython-313-pytest-9.0.3.pyc matches
Binary file tests/__pycache__/test_api_endpoints.cpython-313-pytest-9.0.3.pyc matches
Binary file tests/__pycache__/test_input_source.cpython-313-pytest-9.0.3.pyc matches
Binary file tests/__pycache__/test_restart_recovery.cpython-313-pytest-9.0.3.pyc matches
Binary file tests/__pycache__/test_storage_gdrive_api.cpython-313-pytest-9.0.3.pyc matches
Binary file tests/__pycache__/test_tracknet_hybrid.cpython-313-pytest-9.0.3.pyc matches
```

25. assistant (2026-07-08T06:17:31.496Z)

Every `_stitch_reel` reference in the test suite is a monkeypatch ? no test calls the real function, so the mirror-upload swallow path has zero coverage. Let me confirm this block is #521's changed code by inspecting the diff, and check whether `storage.put` retries could make the transient case non-fatal.

26. user (2026-07-08T06:17:38.261Z)

```
===== diff dac6736^..dac6736 for _stitch_reel region in orchestration.py =====
220:+def _stitch_reel(
274:+    # still falls back to the local file on the same instance). On the L4 worker the mirror
is how the
```

```
327:+        return _stitch_reel(
466:+    return _stitch_reel(
```

27. user (2026-07-08T06:17:42.738Z)

```
===== was _stitch_reel a rename/extraction? Check the - side (pre-521) for the mirror swallow
=====
193:-def _execute_compile(cfg, db, job: Dict[str, Any], progress=None) -> Dict[str, Any]:
```

28. user (2026-07-08T06:17:51.714Z)

```
(Bash completed with no output)
```

29. user (2026-07-08T06:17:58.874Z)

```
(Bash completed with no output)
```

30. user (2026-07-08T06:18:10.001Z)

```
468 C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-khelsutra-guru-badminton-
highlight-indexer\75d3d121-f240-4293-afa5-b436a379ad31\scratchpad/orch_521.diff
19:    - compile ? L4: a kind='compile' Cloud Run dispatch mirroring _maybe_dispatch_remote.
140:         # ``force=true`` bypasses the reuse and re-writes a fresh verdict below (mirrors
how it bypasses
200:+    """The live job.progress line while an L4 compile/stitch is in flight (#521; mirrors
230:+    """Stitch + mirror-upload core, shared by the control-plane compile handler AND the L4
worker
234:+    trim/watermark/concat, mirrors the reel to
``<output_root>/highlights/<video_id>/<name>``, and
270:     # #463: mirror the reel to the output bucket so /api/highlights can 307 to a signed URL
? Cloud Run
272:     # instance recycles. A mirror-upload failure must NOT fail an otherwise-good stitch
(the endpoint
274:+    # still falls back to the local file on the same instance). On the L4 worker the mirror
is how the
367:+    # Forward the bucket coordinates so the worker (no DEPLOYMENT_PROFILE) can read the
spec, mirror the
368:+    # reel, and write the marker ? mirrors how _maybe_dispatch_remote forwards the WASB
knobs for detect.
```

31. user (2026-07-08T06:18:13.513Z)

```
255              # same seam the process path uses); without it, compiling a bucket-ingested
video can't open gs://.
256              local_src = storage.acquire_local_input(
257     -            video["filepath"], getattr(cfg, "storage", None)
258     +            source_ref, getattr(cfg, "storage", None)
259              )
260              # The ffmpeg stitch is CPU-heavy ? serialize it so a raised drain worker count
(for remote
261              # dispatch) doesn't run multiple compiles at once on the box (see
_LOCAL_COMPUTE_LANE).
262     @@ -1175,13 +1245,14 @@ def _execute_compile(cfg, db, job: Dict[str, Any],
progress=None) -> Dict[str, A
263          finally:
264              if local_src:
265                  storage.cleanup_acquired_local_input(
266     -                video["filepath"], local_src, getattr(cfg, "storage", None)
267     +                source_ref, local_src, getattr(cfg, "storage", None)
268                  )
269
270          # #463: mirror the reel to the output bucket so /api/highlights can 307 to a signed
URL ? Cloud Run
271          # caps a buffered response at ~32 MiB (a real reel is bigger) and the local /tmp
```

```
copy dies when the
272          # instance recycles. A mirror-upload failure must NOT fail an otherwise-good stitch
(the endpoint
273      -    # still falls back to the local file on the same instance).
274      +    # still falls back to the local file on the same instance). On the L4 worker the
mirror is how the
275      +    # reel reaches the bucket at all (the worker's local file is discarded when the Job
exits).
276          reel_uri = reel_object_uri(cfg, video_id, out_name)
277          if reel_uri:
278              try:
279      @@ -1201,4 +1272,189 @@ def _execute_compile(cfg, db, job: Dict[str, Any],
progress=None) -> Dict[str, A
280              "filepath": out_path,
281              "download_url": download_url,
282              "video_id": video_id,
283      +        "reel_uri": reel_uri or "",
284          }
285      +
286      +
287      +def _dispatch_compile_remote(
288      +    cfg,
289      +    db,
290      +    video_id: str,
291      +    segment_ids: List[int],
292      +    output_filename: str,
293      +    locale: Optional[str],
294      +    notify,
295      +) -> Dict[str, Any]:
296      +    """#521: run the compile/stitch on the L4 worker instead of the CPU-only control-
plane. The
297      +    control-plane owns the DB, so it RESOLVES the reel's segments here
(``_resolve_compile``), stages a
298      +    small compile-spec (resolved time-pairs + the resolved stitching/watermark config +
the source URI)
299      +    to the bucket, dispatches a ``compile`` Cloud Run Job on the L4 (NVENC), bucket-
polls the
300      +    done-marker, and returns the same ``{status, filepath, download_url}`` shape the
SPA expects. The
301      +    worker needs NO control-plane DB ? everything it needs rides the spec. Mirrors
302      +    ``_maybe_dispatch_remote`` (the DETECT dispatch)."""
303      +    from backend.compute import CompileJobSpec, get_compute_backend
304      +
305      +    notify("resolving")
306      +    try:
307      +        video, kept, out_name = _resolve_compile(
308      +            db, cfg, video_id, segment_ids, output_filename
309      +        )
310      +    except _CompileError as e:
311      +        return {"status": "failed", "message": e.detail, "video_id": video_id}
312      +
313      +    stitching = getattr(cfg, "stitching", None) or StitchingConfig()
314      +    storage_cfg = getattr(cfg, "storage", None)
315      +    out_root = (getattr(storage_cfg, "output_root", "") or "").strip()
316      +    compute = get_compute_backend(cfg, target_override="cloud_run") if out_root else
None
317      +    if compute is None:
318      +        # Compile was placed on the L4 but the cloud dispatch isn't actually wired here
(no output
319      +        # bucket / Cloud Run env). Fall back to the in-process stitch rather than fail
? a slow reel
320      +        # beats no reel. (The placement guard means this is only reachable in a mis-
provisioned setup.)
321      +        logger.warning(
322      +            "[compile] compile placed on the L4 but no cloud backend available
```

```
(out_root=%r) ? "
323     +            "stitching in-process on the control-plane instead.",
324     +            out_root,
```

32. assistant (2026-07-08T06:18:58.331Z)

I have now traced the entire failure chain in the actual merged #521 code (commit `dac6736`). Let me record my verdict.

I tried hard to refute this claim and could not. Every link in the chain is concretely confirmed:

The behavioral failure mode is real (silent data loss on the L4):
- `_stitch_reel` (new in #521, `orchestration.py:1199`) mirrors the reel via `storage.put(out_path, reel_uri, ...)` inside a `try/except` at `orchestration.py:1257-1264` that swallows any exception (logs a warning) and then returns `{"status": "success", ...}` at lines 1270-1276.
- #521 itself added the comment at `orchestration.py:1254-1255` explicitly acknowledging the new hazard: *"On the L4 worker the mirror is how the reel reaches the bucket at all (the worker's local file is discarded when the Job exits)."* ? yet keeps the swallow-and-report-success behavior. On the control-plane (#463) the swallow is benign (local fallback); on the L4 there is no fallback, so it becomes silent data loss.
- The worker's `cmd_compile` sets `rc = 0 if status == "success" else 3` (`worker/__main__.py:514`) with **no `storage.exists` verification** of the uploaded reel, then writes a SUCCESS done-marker (`worker/__main__.py:519-520`).
- The dispatcher `_dispatch_compile_remote` returns `{status: success, download_url}` on `result.returncode == 0` (`orchestration.py:1391-1420`) with **no reel-existence check**.
- `/api/highlights` (`main.py:1060-1083`): `signed_reel_url` ? `storage.exists(reel_uri)` is False ? `signed` is None ? falls back to the local file at `output_dir/name`, which does not exist on the control-plane (the stitch ran on the worker) ? HTTP 404. Confirmed.

The test-coverage gap is real: Every reference to `_stitch_reel` in the test suite is a monkeypatch replacement (`test_worker.py:1015, 1062, 1100`; `test_compile_dispatch.py:221`). No test calls the real `_stitch_reel`, so the mirror-upload `storage.put`-failure path (orchestration.py:1257-1264) has zero coverage. The dispatch tests use a fake `CloudRunJobClient` (`test_compile_dispatch.py:25`) that never runs the real worker mirror.

Refutation attempts that all failed: no local fallback on the L4 (comment admits it), no post-upload `storage.exists` verification in the worker or dispatcher, no SPA retry, and `storage.put` can raise on non-retryable errors (permissions/quota/bucket-misconfig) even if transient 5xx are retried. The block is genuinely #521's changed/activated code (`_stitch_reel` is new and newly invoked by the L4 worker).

33. user (2026-07-08T06:19:08.774Z)

Structured output provided successfully